Universidade Federal da Fronteira Sul
Curso de Ciência da Computação
Campus Chapecó

Circuitos Aritméticos

Prof. Luciano L. Caimi
lcaimi@uffs.edu.br

Circuitos Aritméticos

Adição Binária

Adição de números sem sinal

	A
+	**B**
	S

	0	1	0	0		Transporte (*carry*)
		0	1	0	0	(4)
+		0	1	1	0	(6)
		1	0	1	0	(10) resultado

O maior número binário que podemos representar com 4 bits é 15

Circuitos Aritméticos

Adição Binária

Adição de números sem sinal

	1	1	0	0		Transporte (*carry*)
		1	1	0	0	(12)
+		0	1	1	0	(6)
		0	0	1	0	(18) resultado

overflow

Circuitos Aritméticos

Adição Binária

Outro exemplo (8 bits)

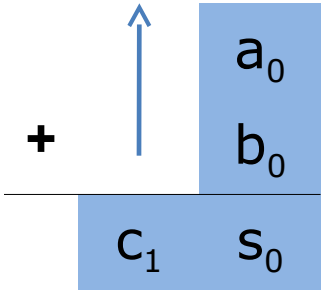
	0	1	1	0	1	1	0	0		
		0	1	1	0	1	1	0	1	(109) → A
+		0	1	1	0	0	1	1	0	(102) → + B
		1	1	0	1	0	0	1	1	(211) → S

Circuitos Aritméticos

Adição Binária

Adição de números sem sinal

	0	1	1	0	1	1	0	0		
		0	1	1	0	1	1	0	1	(109)
+		0	1	1	0	0	1	1	0	(102)
		1	1	0	1	0	0	1	1	(211)



adição de 2 bits

resultado em 2 bits

Circuitos Aritméticos

Adição Binária

A partir do 2º bit

	0	1	1	0	1	1	0	0	
		0	1	1	0	1	1	0	1	(109)
+		0	1	1	0	0	1	1	0	(102)
		1	1	0	1	0	0	1	1	(211)

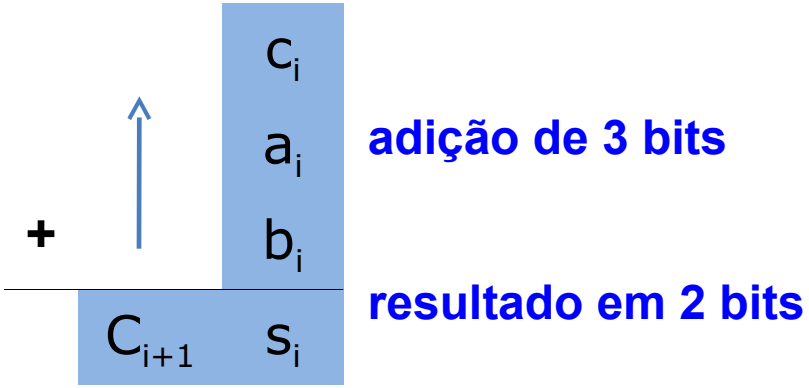
	c_1	
	a_1	adição de 3 bits
+	b_1	
c_2	s_1	resultado em 2 bits

Circuitos Aritméticos

Adição Binária

Generalizando, a partir do 2º bit

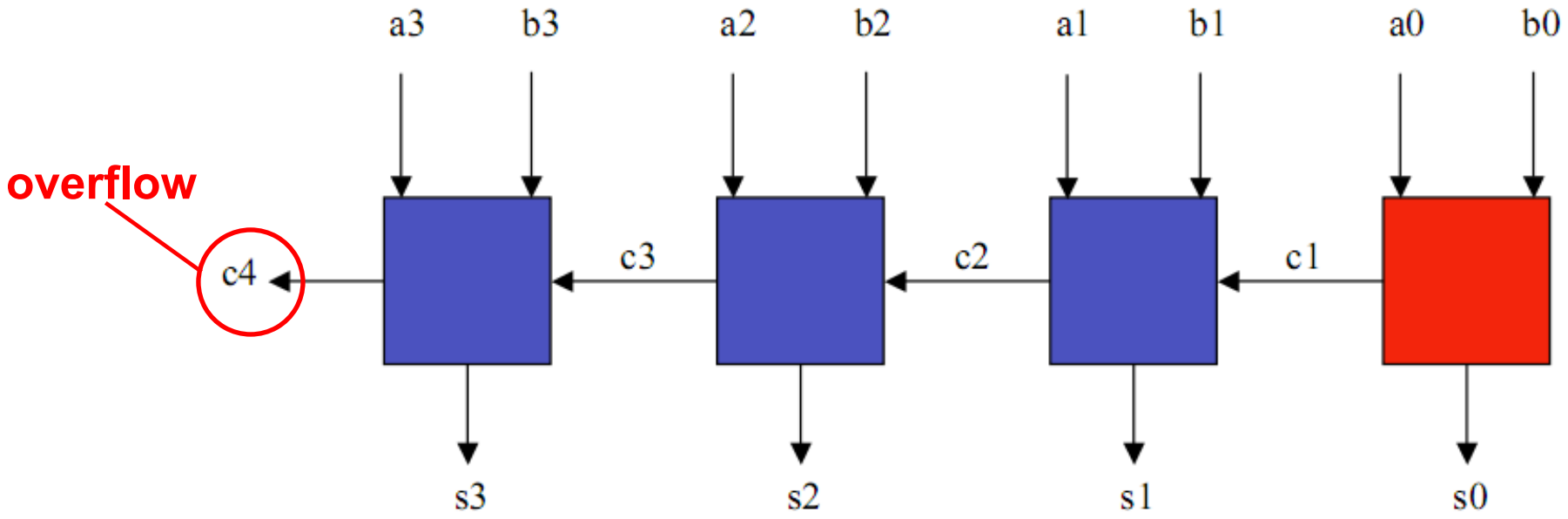
	0	1	1	0	1	1	0	0	
		0	1	1	0	1	1	0	1 (109)
+		0	1	1	0	0	1	1	0 (102)
		1	1	0	1	0	0	1	1 (211)



Circuitos Aritméticos

Adição Binária

Esquema para soma paralela



Observe:

- Existe um elemento para cada coluna da soma
- O sinal de *overflow* será o *carry* mais significativo

Circuitos Aritméticos

Adição Binária

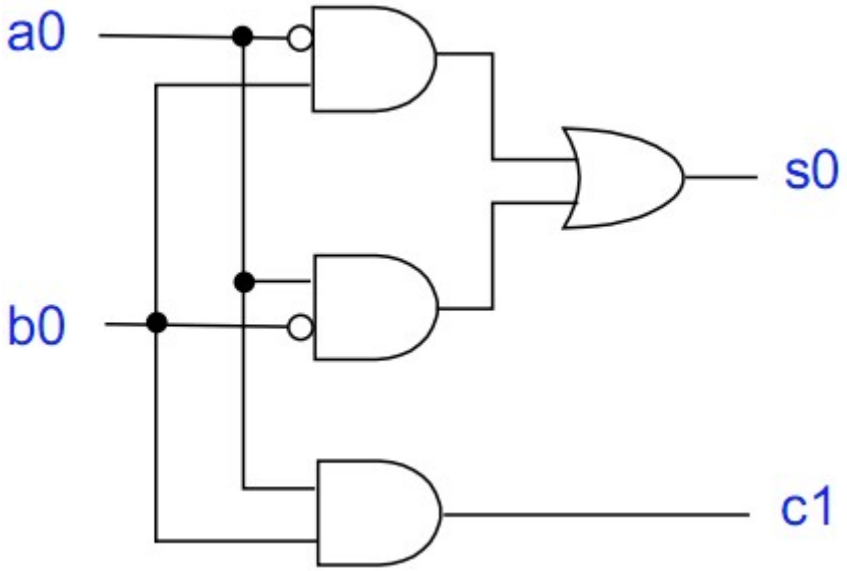
Projetando o circuito para primeira coluna:

Meio-Somador (Half-Adder)

$$s_0 = \overline{a_0}.b_0 + a_0.\overline{b_0}$$

$$c_1 = a_0.b_0$$

entradas		saídas	
a_0	b_0	c_1	s_0
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

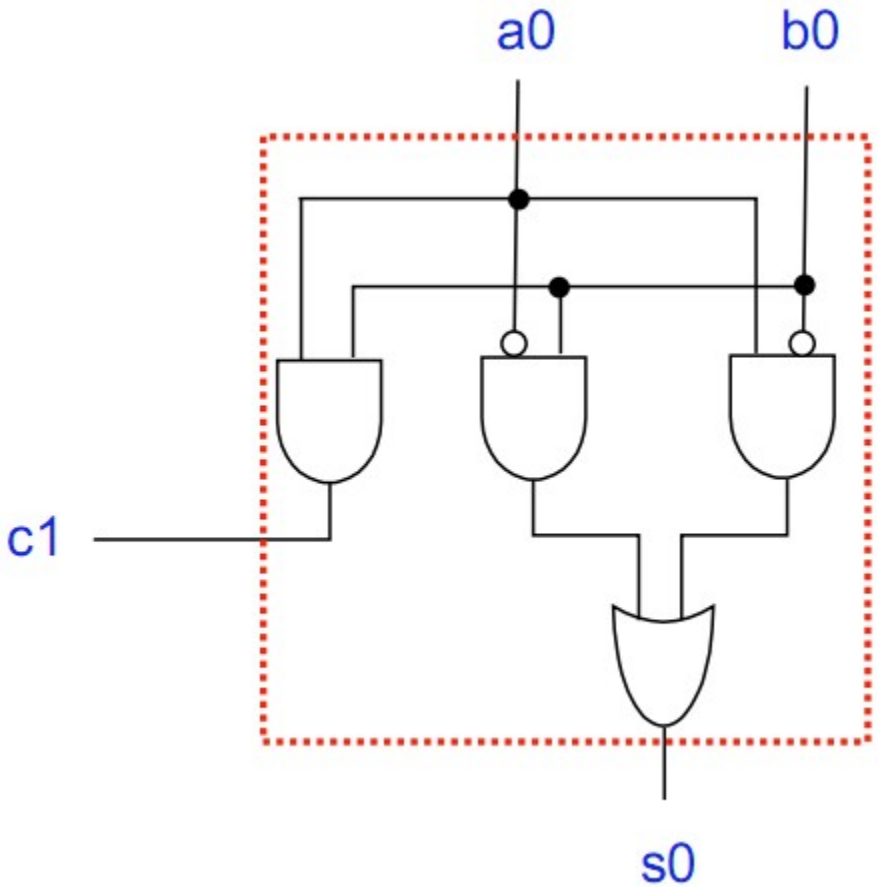


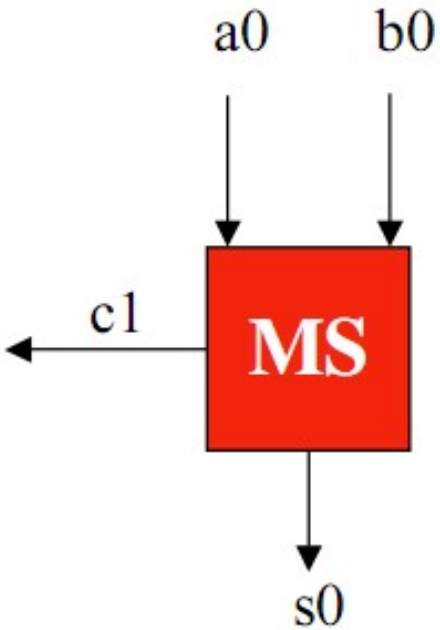
Circuitos Aritméticos

Adição Binária

Meio-Somador (Half-Adder)

...se preferir





$$s_0 = \overline{a_0}.b_0 + a_0.\overline{b_0}$$

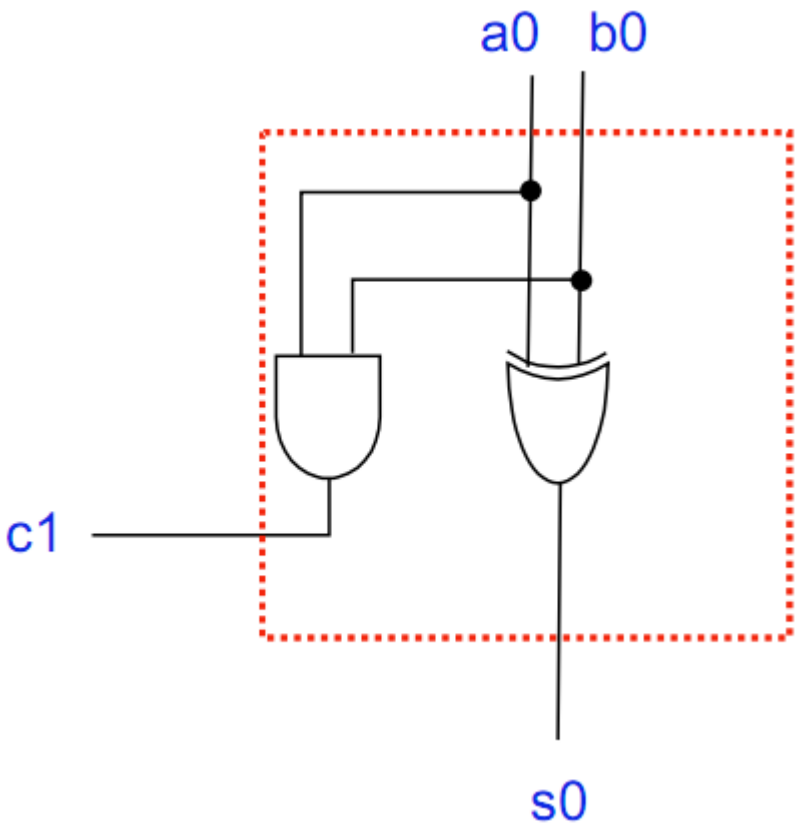
$$c_0 = a_0.b_0$$

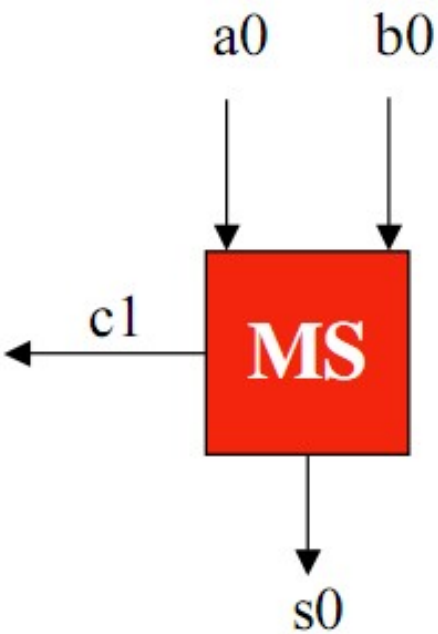
Circuitos Aritméticos

Adição Binária

Meio-Somador (Half-Adder)

...ou ainda





$$s_0 = \overline{a_0}.b_0 + a_0.\overline{b_0}$$

$$a_0 \oplus b_0$$

$$c_0 = a_0.b_0$$

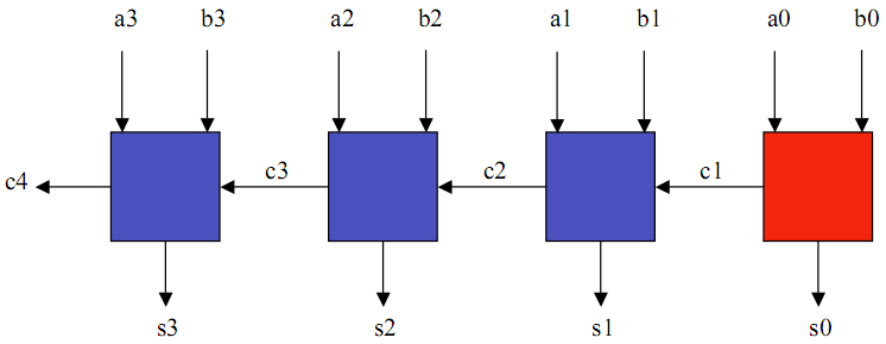
Circuitos Aritméticos

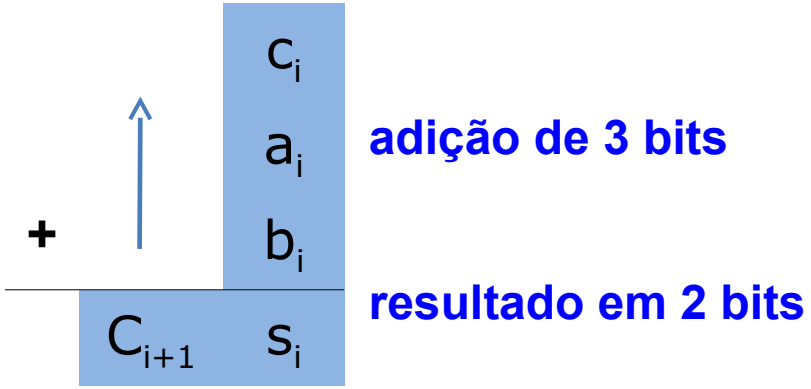
Adição Binária

Projetando as demais colunas:

Somador-Completo (Full-Adder)

	0	1	1	0	1	1	0	0	
		0	1	1	0	1	1	0	1
+		0	1	1	0	0	1	1	0
		1	1	0	1	0	0	1	1





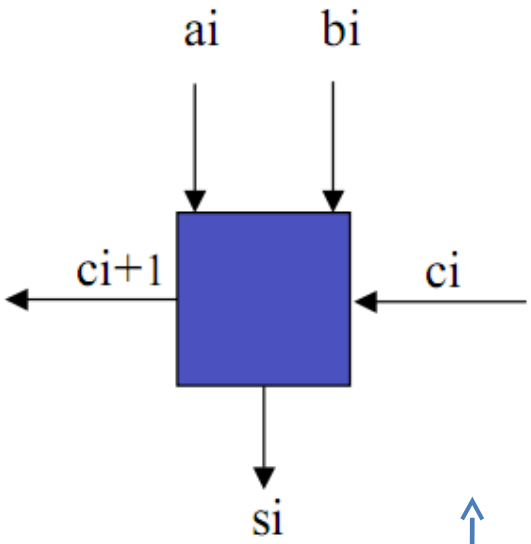
Circuitos Aritméticos

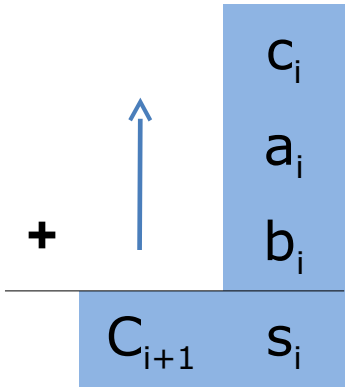
Adição Binária

Projetando as demais colunas:

Somador-Completo (Full-Adder)

entradas			saídas	
a_i	b_i	c_i	C_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1





adição de 3 bits

resultado em 2 bits

Circuitos Aritméticos

Adição Binária

Projetando as demais colunas:

Somador-Completo (Full-Adder)

entradas			saídas	
a_i	b_i	c_i	C_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Mapa de Karnaugh para c_{i+1}

c_{i+1}

	$\overline{b_i}$		b_i	
$\overline{a_i}$	0	0	1	0
a_i	0	1	1	1
	$\overline{c_i}$		c_i	$\overline{c_i}$

$$c_{i+1} = a_i . b_i + a_i . c_i + b_i . c_i$$

Adição Binária

Projetando as demais colunas:

Somador-Completo (Full-Adder)

entradas			saídas	
a_i	b_i	c_i	C_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Mapa de Karnaugh para s_i

s_i	$\overline{b_i}$		b_i	
$\overline{a_i}$	0	1	0	1
a_i	1	0	1	0
	$\overline{c_i}$	c_i		$\overline{c_i}$

Não é possível simplificar, logo:

$$s_i = \overline{a_i}.\overline{b_i}.c_i + \overline{a_i}.b_i.\overline{c_i} + a_i.\overline{b_i}.\overline{c_i} + a_i.b_i.c_i$$

Circuitos Aritméticos

Adição Binária

manipulando através da álgebra de boole:

$$s_i = \overline{a}_i . \overline{b}_i . c_i + \overline{a}_i . b_i . \overline{c}_i + a_i . \overline{b}_i . \overline{c}_i + a_i . b_i . c_i$$

$$s_i = \overline{a}_i . \left(\overline{b}_i . c_i + b_i . \overline{c}_i \right) + a_i . \left(\overline{b}_i . \overline{c}_i + b_i . c_i \right)$$

$$s_i = \overline{a}_i . \left(b_i \oplus c_i \right) + a_i . \left(\overline{b_i \oplus c_i} \right)$$

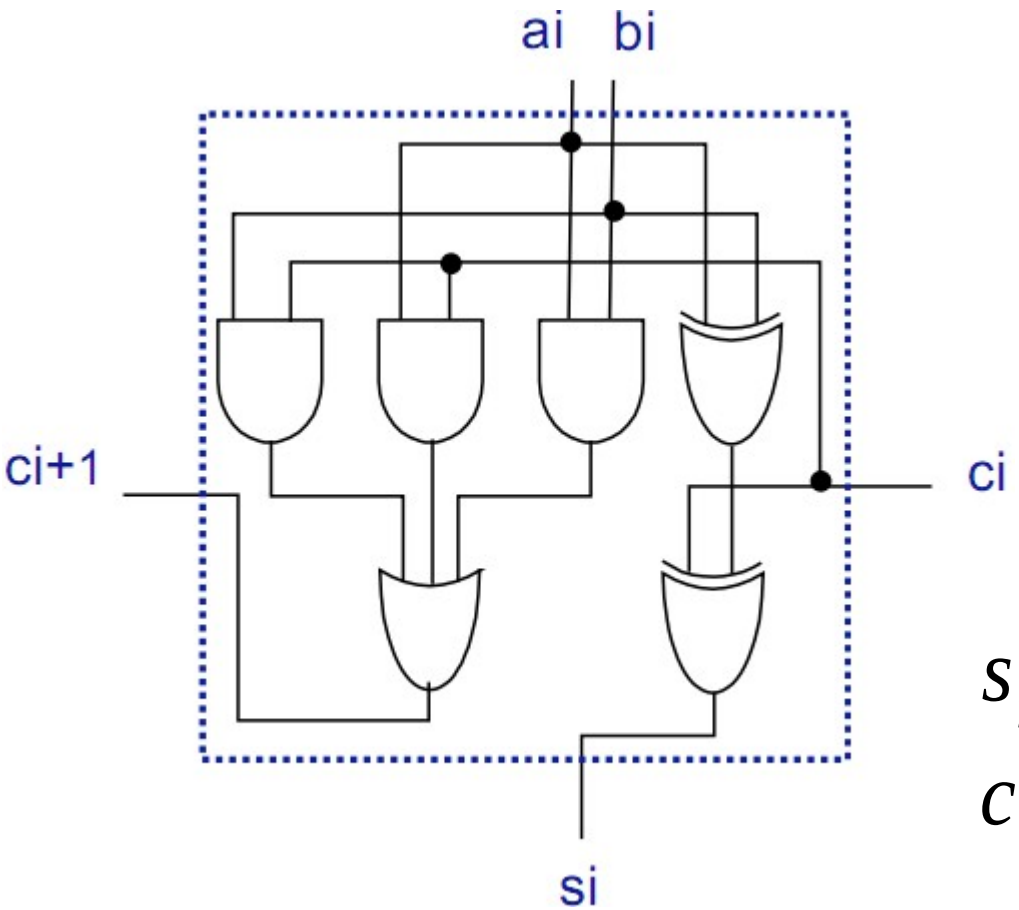
$$s_i = a_i \oplus \left(b_i \oplus c_i \right)$$

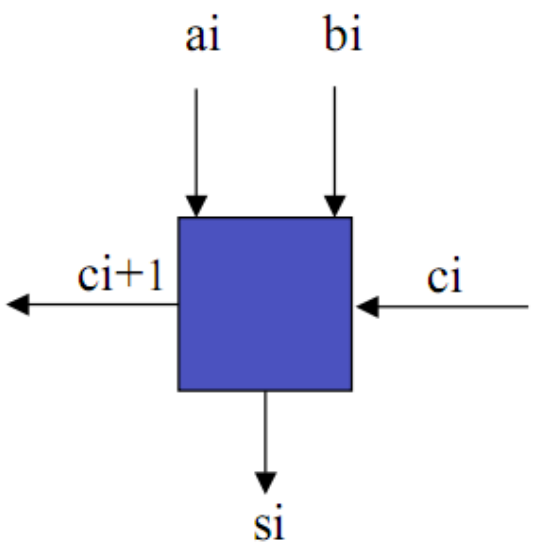
a	b	xor	$\overline{\text{xor}}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

Circuitos Aritméticos

Adição Binária

Somador-Completo (Full-Adder)





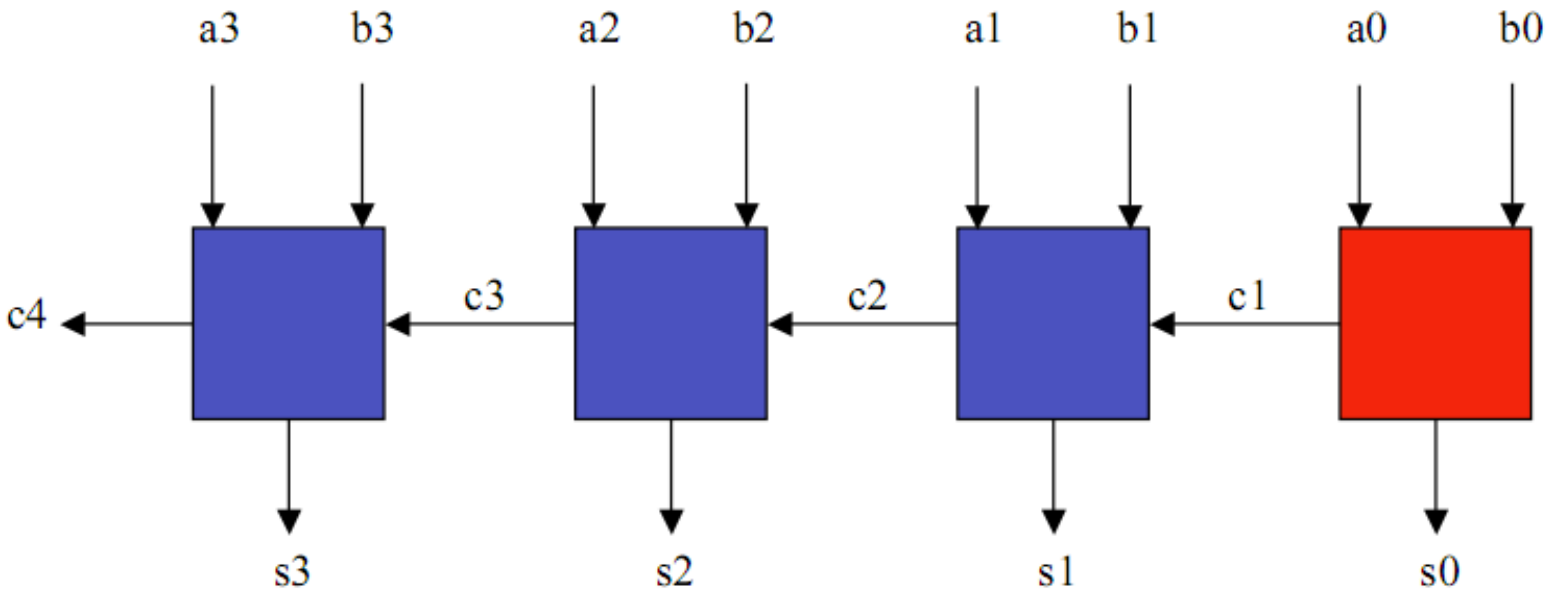
$$s_i = a_i \oplus (b_i \oplus c_i)$$

$$c_{i+1} = a_i . b_i + a_i . c_i + b_i . c_i$$

Circuitos Aritméticos

Adição Binária

Considerando dois números (A e B) de 4 bits cada

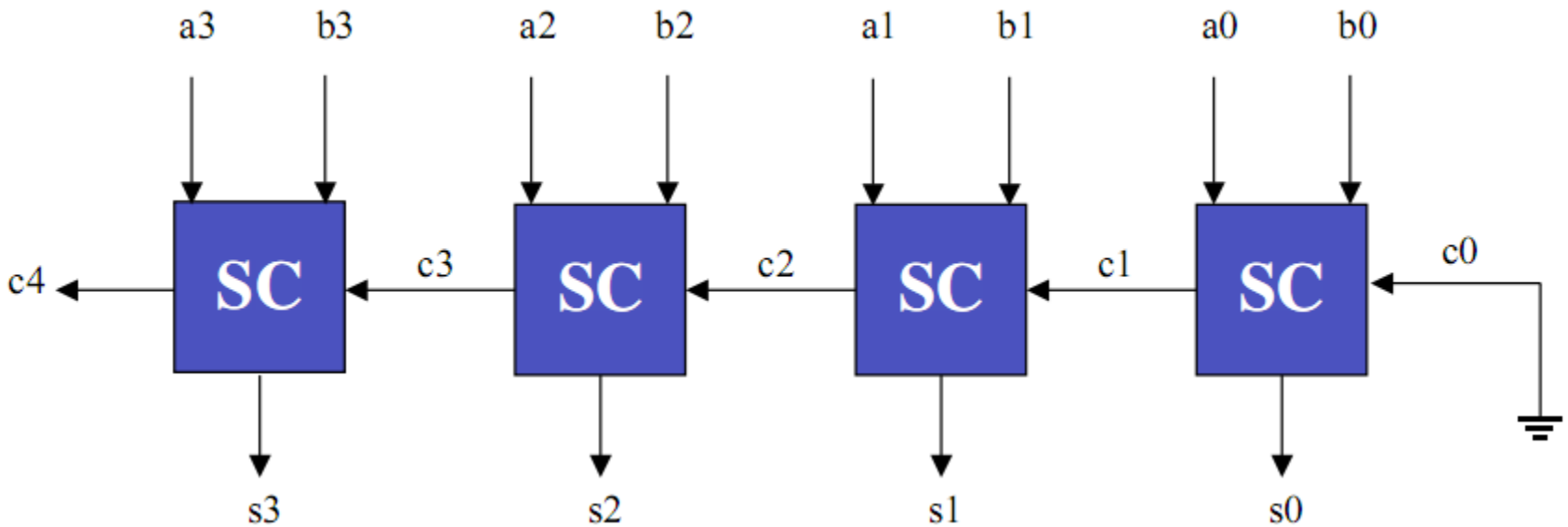


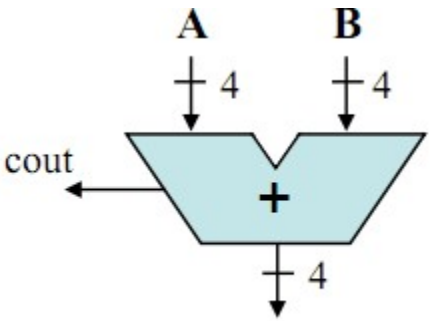
Chamado somador *ripple-carry*

Circuitos Aritméticos

Adição Binária

Versão utilizando somente somador-completo





Custo é ligeiramente maior.

Circuitos Aritméticos

Adição de números inteiros

E se quisermos realizar operações sobre números com sinal???

- Precisaremos considerar uma representação que sirva tanto para binários positivos quanto binários negativos
- A representação mais usada, neste caso, é complemento de 2
- Porque mesmo?!?!?!?

Adição de números inteiros

Representação em complemento de 2

Sinal	Regra de formação	exemplo
positivo	igual a sinal-maginitude	+13 = 00001101
negativo	1. toma-se a representação em sinal e magnitude 2. faz-se o complemento de 1 (inverte o número bit a bit) 3. soma-se 1	+13 = 00001101 11110010 + 1 -13 = 11110011

-128	64	32	16	8	4	2	1
1	1	1	1	0	0	1	1

Adição de números inteiros

Adição de números em complemento de 2

Considere valor binários de 4 bits, portanto, com faixa de representação de [-8:+7]

Exemplo 1: números positivos com soma ≤ 7

	0	0	0	0		Transporte (*carry*)
		0	1	0	0	(+4)
+		0	0	1	0	(+2)
		0	1	1	0	(+6) Resultado correto

Adição de números inteiros

Adição de números em complemento de 2

Exemplo 2: nros negativos com soma ≥ -8

	1	1	0	0		**Transporte (*carry*)**
		1	1	0	0	(-4)
+		1	1	1	0	(-2)
		1	0	1	0	(-6) **Resultado correto**

Apesar deste último carry valer 1, não houve "overflow"

Adição de números inteiros

Adição de números em complemento de 2

Exemplo 3: um número positivo e outro negativo com resultado positivo

	1	1	0	0		Transporte (*carry*)
		0	1	0	0	(+4)
+		1	1	1	0	(-2)
		0	0	1	0	(+2) Resultado correto

Novamente...

Adição de números inteiros

Adição de números em complemento de 2

Exemplo 4: um número positivo e outro negativo com resultado negativo

	0	0	1	0		Transporte (*carry*)
		0	0	1	1	(+3)
+		1	0	1	0	(-6)
		1	1	0	1	(-3) Resultado correto

Adição de números inteiros

Adição de números em complemento de 2

Exemplo 5: um número positivo e outro negativo iguais em módulo

	1	1	1	1		Transporte (*carry*)
		0	1	0	1	(+5)
+		1	0	1	1	(-5)
		0	0	0	0	(-0) Resultado correto

Adição de números inteiros

Adição de números em complemento de 2

Exemplo 6: dois números positivos

	0	1	0	0		Transporte (*carry*)
		0	1	0	1	(+5)
+		0	1	0	0	(+4)
		1	0	0	1	(+9)

Resultado Errado

Em complemento de 2 o resultado foi -7

Adição de números inteiros

Adição de números em complemento de 2

Exemplo 7: dois números negativos

	1	0	0	0		Transporte (*carry*)
		1	1	0	0	(-4)
+		1	0	1	1	(-5)
		0	1	1	1	(-9)

Resultado Errado

Em complemento de 2 o resultado foi 7

Circuitos Aritméticos

Adição de números inteiros

Resultados Corretos

	0	0	0	0	
	0	1	0	0	(+4)
+	0	0	1	0	(+2)
	0	1	1	0	(+6)

	1	1	0	0	
	1	1	0	0	(-4)
+	1	1	1	0	(-2)
	1	0	1	0	(-6)

	0	0	1	0	
	0	0	1	1	(+3)
+	1	0	1	0	(-6)
	1	1	0	1	(-3)

	1	1	1	1	
	0	1	0	1	(+5)
+	1	0	1	1	(-5)
	0	0	0	0	(-0)

Resultados Errados

	0	1	0	0	
	0	1	0	1	(+5)
+	0	1	0	0	(+4)
	1	0	0	1	(+9)

	1	0	0	0	
	1	1	0	0	(-4)
+	1	0	1	1	(-5)
	0	1	1	1	(-9)

Circuitos Aritméticos

Adição de números inteiros

Resultados Corretos

	0	0	0	0	
	0	1	0	0	(+4)
+	0	0	1	0	(+2)
	0	1	1	0	(+6)

	1	1	0	0	
	1	1	0	0	(-4)
+	1	1	1	0	(-2)
	1	0	1	0	(-6)

	0	0	1	0	
	0	0	1	1	(+3)
+	1	0	1	0	(-6)
	1	1	0	1	(-3)

	1	1	1	1	
	0	1	0	1	(+5)
+	1	0	1	1	(-5)
	0	0	0	0	(-0)

Resultados Errados

	0	1	0	0	
	0	1	0	1	(+5)
+	0	1	0	0	(+4)
	1	0	0	1	(+9)

	1	0	0	0	
	1	1	0	0	(-4)
+	1	0	1	1	(-5)
	0	1	1	1	(-9)

Adição de números inteiros

Adição de números em complemento de 2

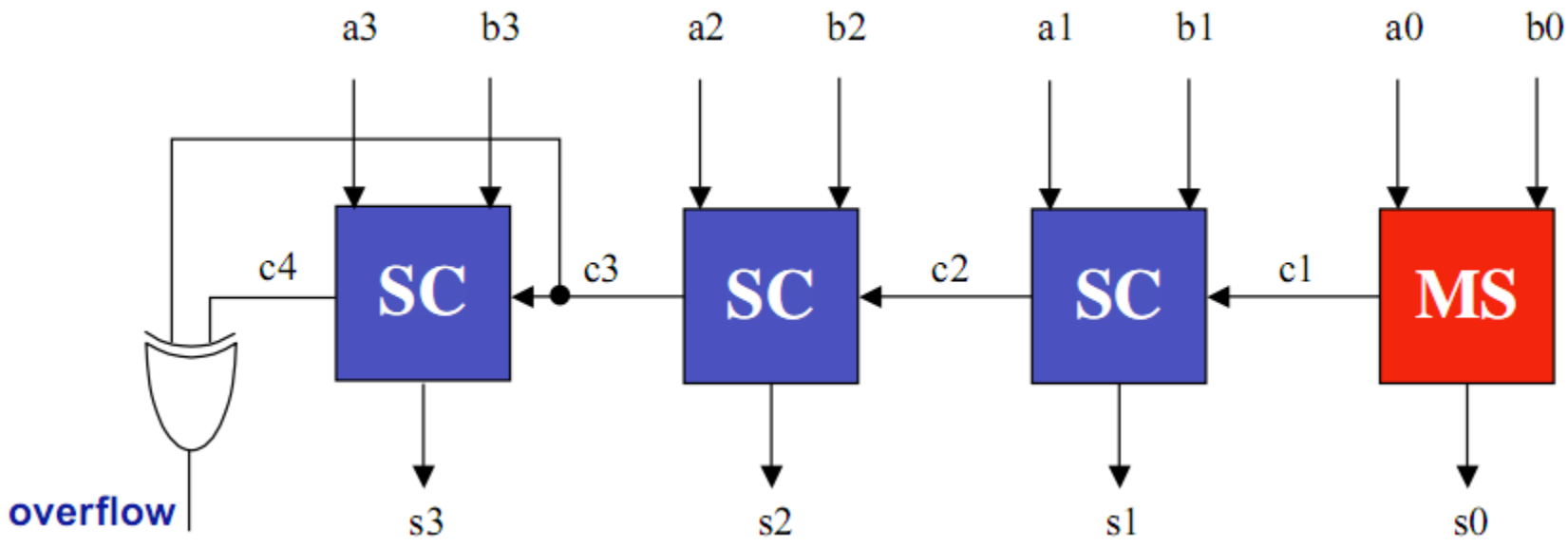
Conclusões:

- Números binários em complemento de 2 podem ser adicionados como se fossem número binários sem sinal
- Neste caso, a detecção de overflow se dá comparando-se os dois últimos sinais de carry

Circuitos Aritméticos

Adição de números inteiros

Adição de números em complemento de 2



esquemático de blocos

Circuitos Aritméticos

▶ Subtração Binária

Princípio Básico

$$A - B = A + (-B)$$

Onde –B é o número B com o sinal trocado

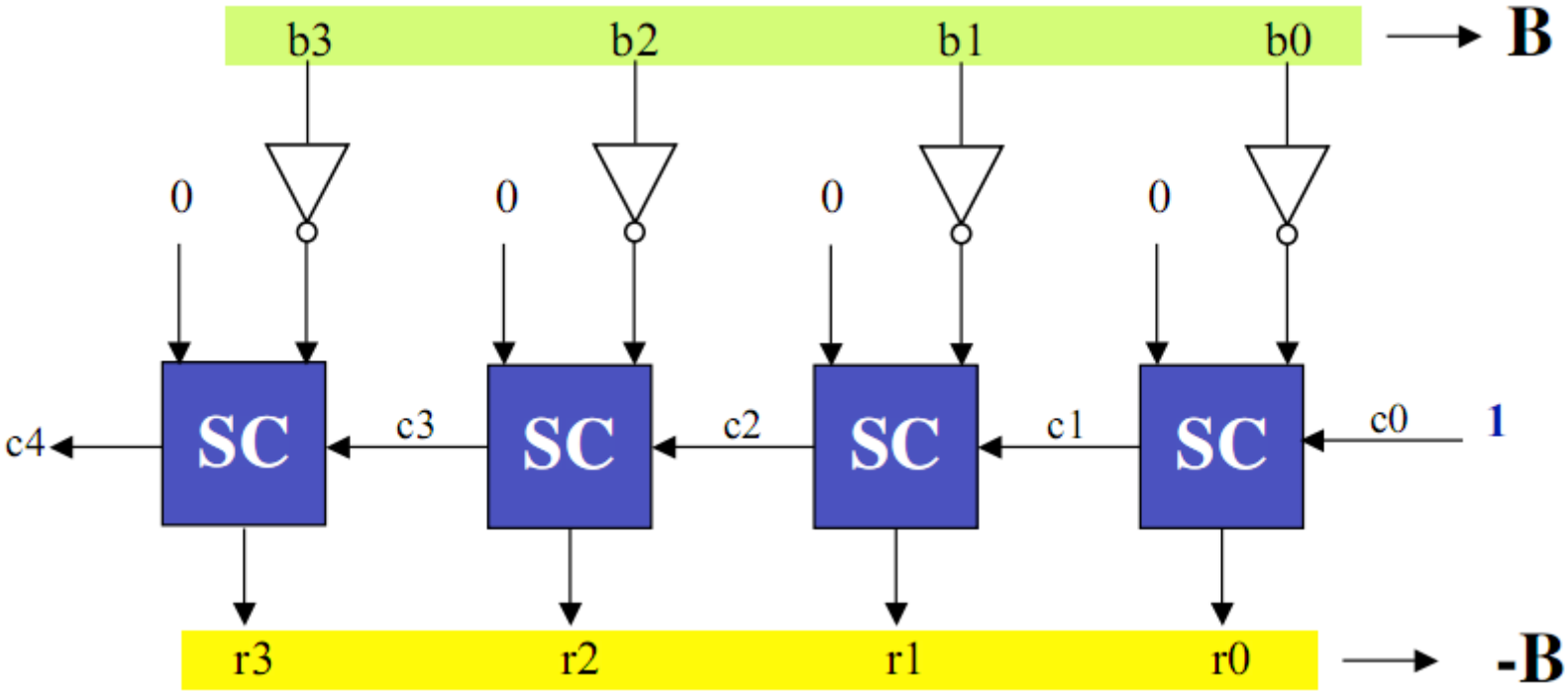
Em complemento de 2:

a) faz o complemento do número

b) soma 1 unidade

Circuitos Aritméticos

Subtração Binária

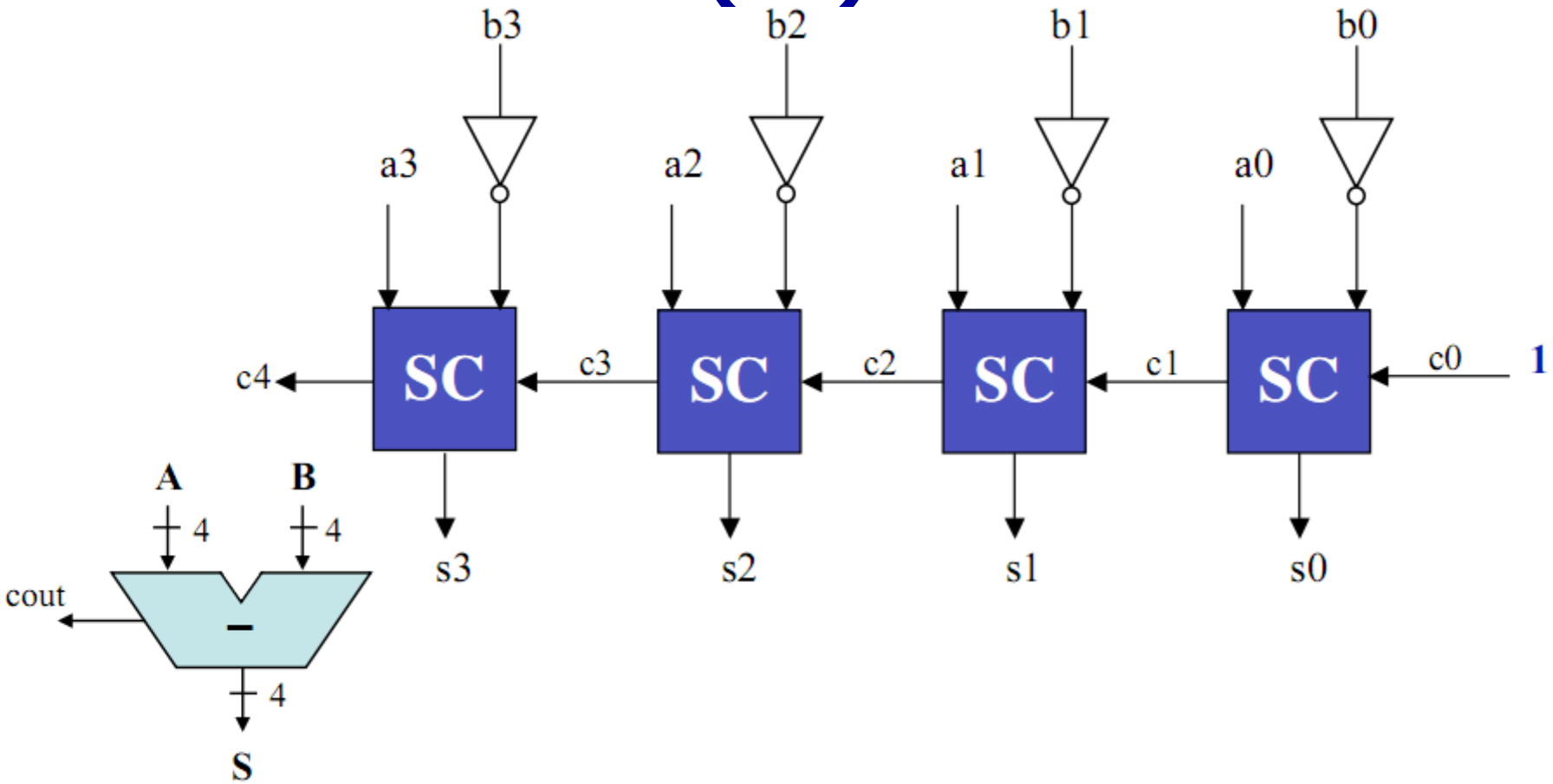


Circuitos Aritméticos

Subtração Binária

Considerando a operação completa

A – B = A + (-B)



Circuitos Aritméticos

Subtração Binária

Mas com tanta semelhança entre os circuitos de adição e subtração não será possível "programar" as 2 operações com o "mesmo" hardware???

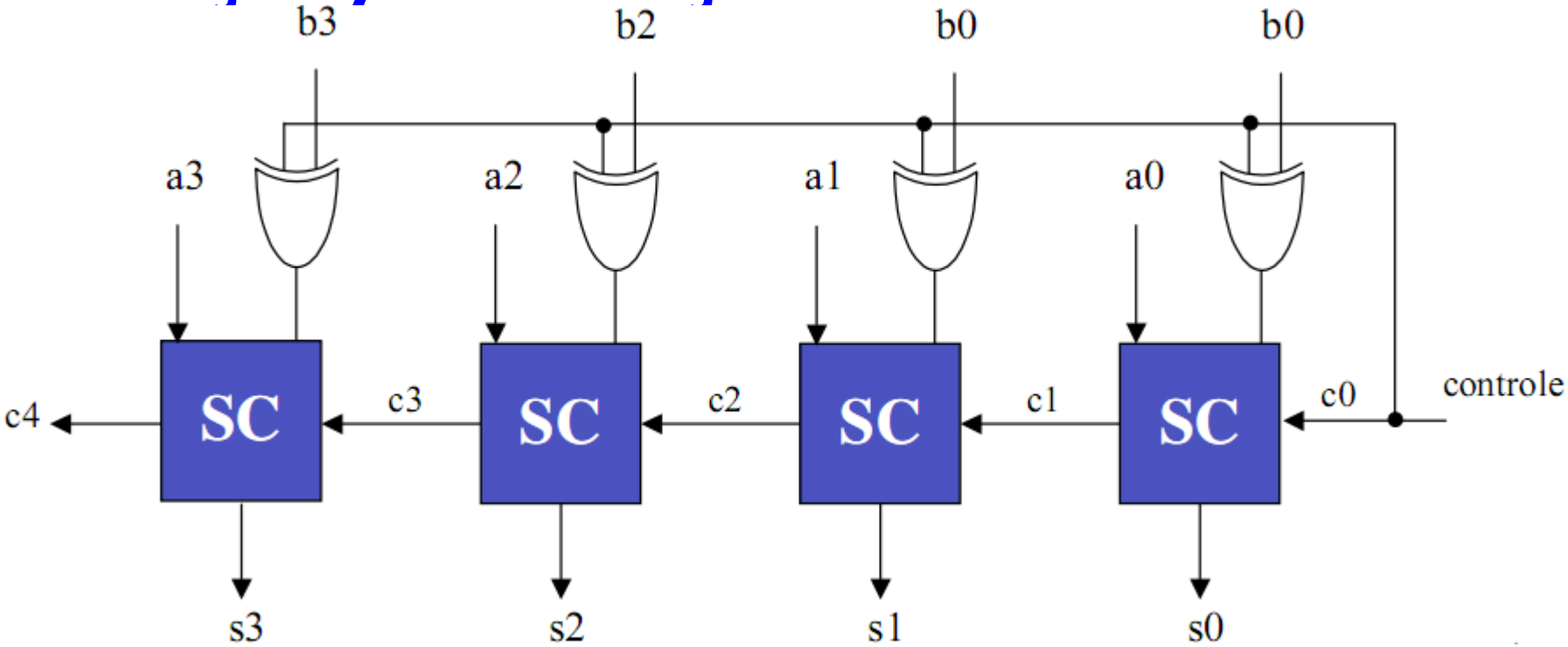
Sim, modificações necessárias:

a) Substituir os inversores por "negadores controlados" (xors);

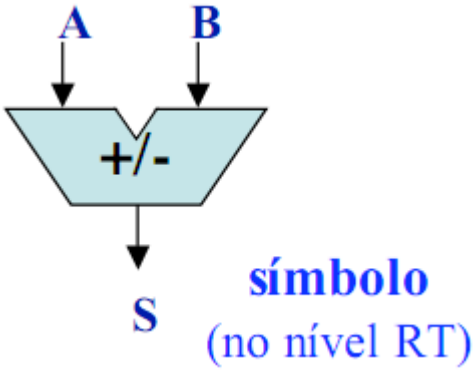
b) Controlar o valor de c0 (0 para adição/1 para subtração)

Circuitos Aritméticos

Adição/Subtração Binária



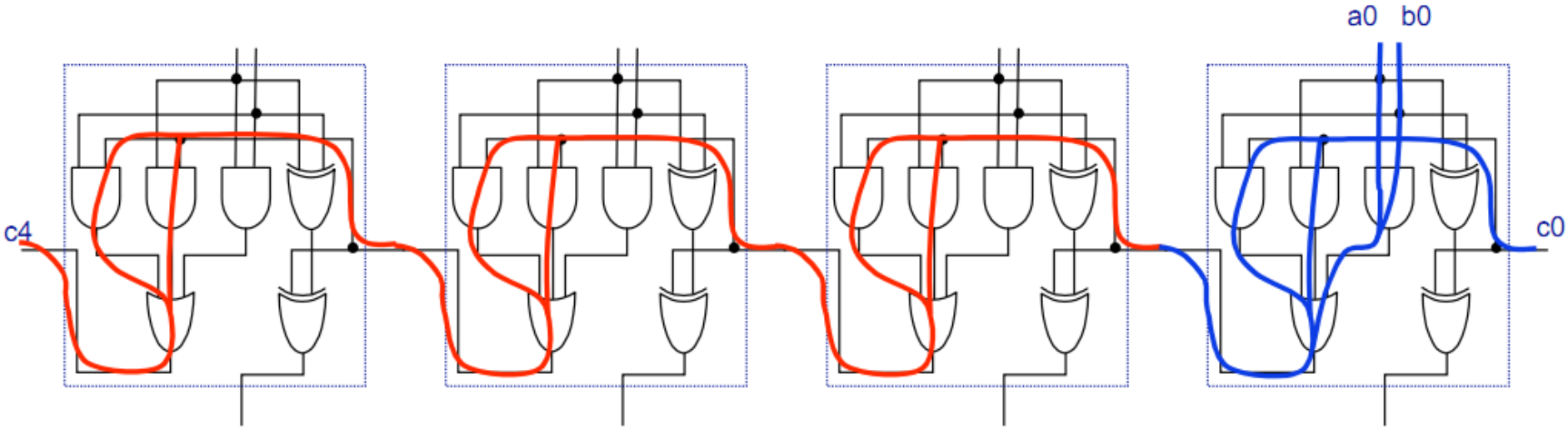
controle	operação
0	S= A + B
1	S= A - B



Circuitos Aritméticos

Analisando o *carry*

Problema da propagação do *carry*



Sinal c_4 estabiliza somente depois de c_1, c_2 e c_3 estabilizarem:

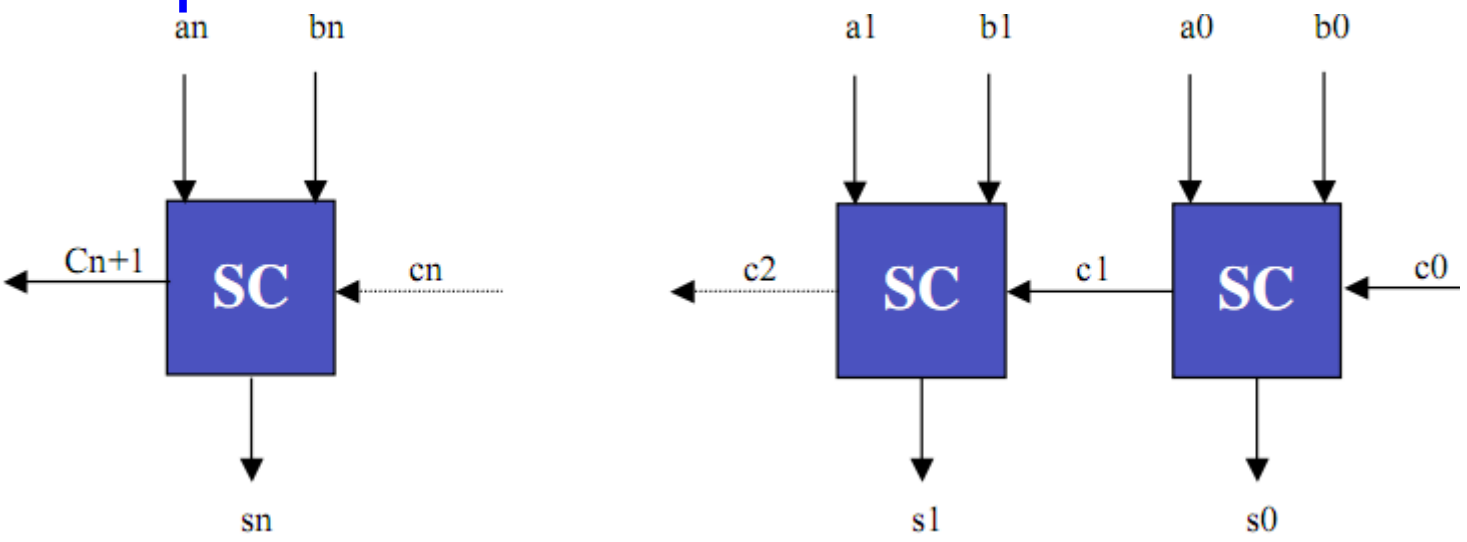
Caminho de maior atraso de a_0 e b_0 ou c_i até c_{i+1}

Caminho de maior atraso de c_i até c_{i+1}

Circuitos Aritméticos

Analisando o *carry*

É possível modificar a arquitetura do somador de modo a "quebrar" ou reduzir a interdependência???



Sim... é possível!!!

Circuitos Aritméticos

Analisando o *carry*

Há três diferentes casos na propagação do *carry* que são mutuamente exclusivos:

a_i	b_i	c_i	k_i	c_{i+1}	s_i
0	0	0	1	0	0
0	0	1	1	0	1
0	1	0	0	0	1
0	1	1	0	1	0
1	0	0	0	0	1
1	0	1	0	1	0
1	1	0	0	1	0
1	1	1	0	1	1

Caso 1 (sinal kill)

Quando as entradas A e B do somador local são iguais a zero, independentemente do *carry* de entrada, o *carry* de saída será igual a zero e, portanto, o estágio "matará" a propagação do *carry* do estágio anterior.

Analisando o *carry*

a_i	b_i	c_i	g_i	c_{i+1}	s_i
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	0	1
0	1	1	0	1	0
1	0	0	0	0	1
1	0	1	0	1	0
1	1	0	1	1	0
1	1	1	1	1	1

Caso 2 (sinal generate)

Quando as entradas A e B do somador local são iguais a um, independentemente do *carry* de entrada, o *carry* de saída será igual a um, portanto, haverá a geração do *carry* neste estágio

Analisando o *carry*

a_i	b_i	c_i	p_i	c_{i+1}	s_i
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	1	0	1
0	1	1	1	1	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	0	1	0
1	1	1	0	1	1

Caso 3 (sinal propagate)

Quando uma das entradas A ou B do somador local for igual a um e a outra igual a zero, o *carry* de saída será igual ao *carry* de entrada e, portanto o *carry* será propagado.

Circuitos Aritméticos

Analisando o *carry*

Resumindo os três casos, temos:

caso	a_i	b_i	c_{i+1}
Kill (k_i)	0	0	0
Propagate (p_i)	0	1	c_i
	1	0	c_i
Generate (g_i)	1	1	1

A idéia é:

- ✓ **encontrar uma equação para cada um dos três sinais (k_i, p_i e g_i)**
- ✓ **achar as equações de s_i e c_{i+1} em função de k_i, p_i e g_i**

Mas qual é a vantagem desta "reestruturação lógica"?

Circuitos Aritméticos

Analisando o *carry*

caso	a_i	b_i	c_{i+1}
Kill (k_i)	0	0	0
Propagate (p_i)	0	1	c_i
	1	0	c_i
Generate (g_i)	1	1	1

As equações:

$$k_i = \overline{a_i} . \overline{b_i} = \overline{a_i + b_i}$$

$$p_i = a_i \oplus b_i$$

$$g_i = a_i . b_i$$

Circuitos Aritméticos

Analisando o *carry*

Partindo da equação original do carry:

$$c_{i+1} = a_i . b_i + a_i . c_i + b_i . c_i$$

$$c_{i+1} = a_i . b_i + (a_i + b_i) . c_i$$

Reescrevendo utilizando *generate* e *propagate*:

$$c_{i+1} = g_i + p_i . c_i$$

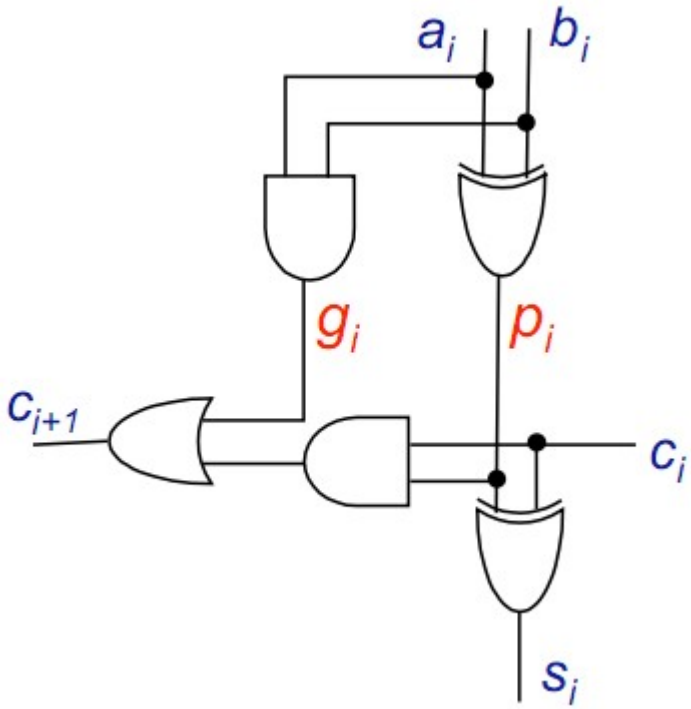
e a saída "soma" do estágio i pode ser expressa em função de c_i e p_i como:

$$s_i = a_i \oplus b_i \oplus c_i = p_i \oplus c_i$$

Circuitos Aritméticos

Analisando o *carry*

As fórmulas de s_i e c_{i+1} geram um circuito formado por dois meio-somadores



$$p_i = a_i \oplus b_i$$

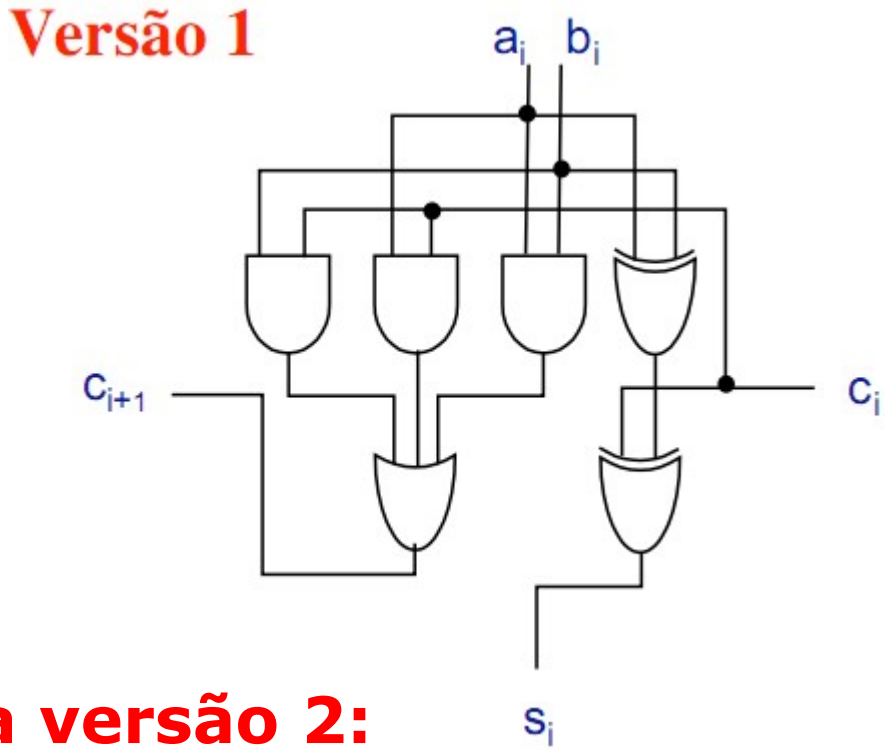
$$g_i = a_i . b_i$$

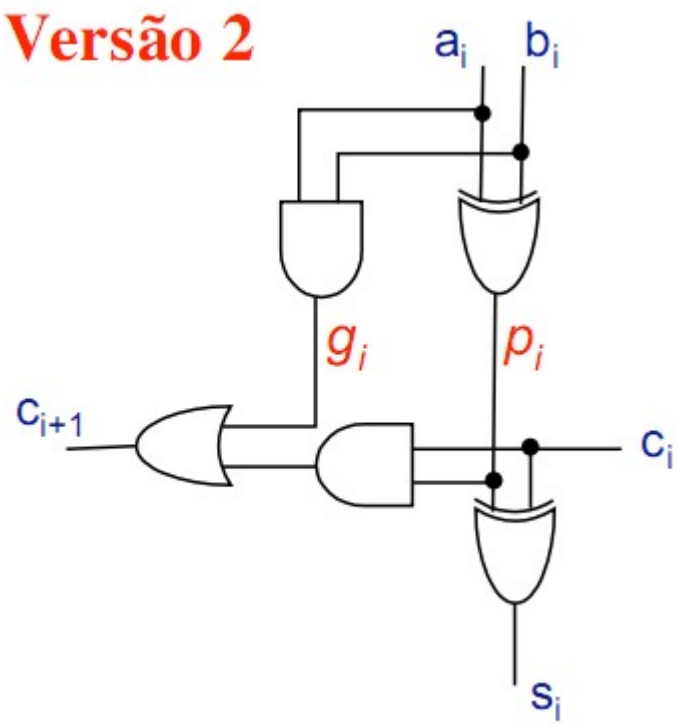
$$c_{i+1} = g_i + p_i . c_i$$

$$s_i = p_i \oplus c_i$$

Circuitos Aritméticos

Comparando as 2 versões





Na versão 2:

- ✓ se $g_i = 1$ não é necessário esperar o valor de c_i para determinar o valor de c_{i+1}
- ✓ se $p_i = 1$ então sabemos a priori que $c_{i+1} = c_i$

Circuitos Aritméticos

Somadores rápidos

Para reduzir o atraso da propagação do carry as seguintes estratégias são utilizadas:

1) **Reduzir o atraso na geração do carry**

   (aplicada nos somadores Manchester)

2) **Diminuir o atraso da cadeia de propagação do carry**

   (aplicada nos somadores *Carry-Lookahead*, *Carry-Select*, *Carry-Skip*, ...)

3) **Mudar o sistema de representação numérica**

   (sistema quaternário, ...)

Estas soluções aumentam o desempenho, mas resultam em aumento dos recursos consumidos

Circuitos Aritméticos

Somadores rápidos

Carry Lookahead

princípio: paralelizar o cálculo dos *carries*

- ✓ *No extremo, todos os carries podem ser computados ao mesmo tempo.*
- ✓ *Para o somador Carry Lookahead, pode-se considerar que não existe mais a exclusividade mútua entre os sinais generate e propagate.*
- ✓ *Então, a função propagate pode ser simplificada para um simples "OU" entre as duas entradas, pois se o nível de soma gera carry out (gi = 1), não importa o valor de propagate.*

Circuitos Aritméticos

Carry Lookahead

Assim, teremos: $p_i = a_i + b_i$

$$c_{i+1} = g_i + p_i . c_i$$

então: $c_1 = g_0 + p_0 . c_0$

$$c_2 = g_1 + p_1 . c_1 = g_1 + p_1 . (g_0 + p_0 . c_0)$$

expandindo:

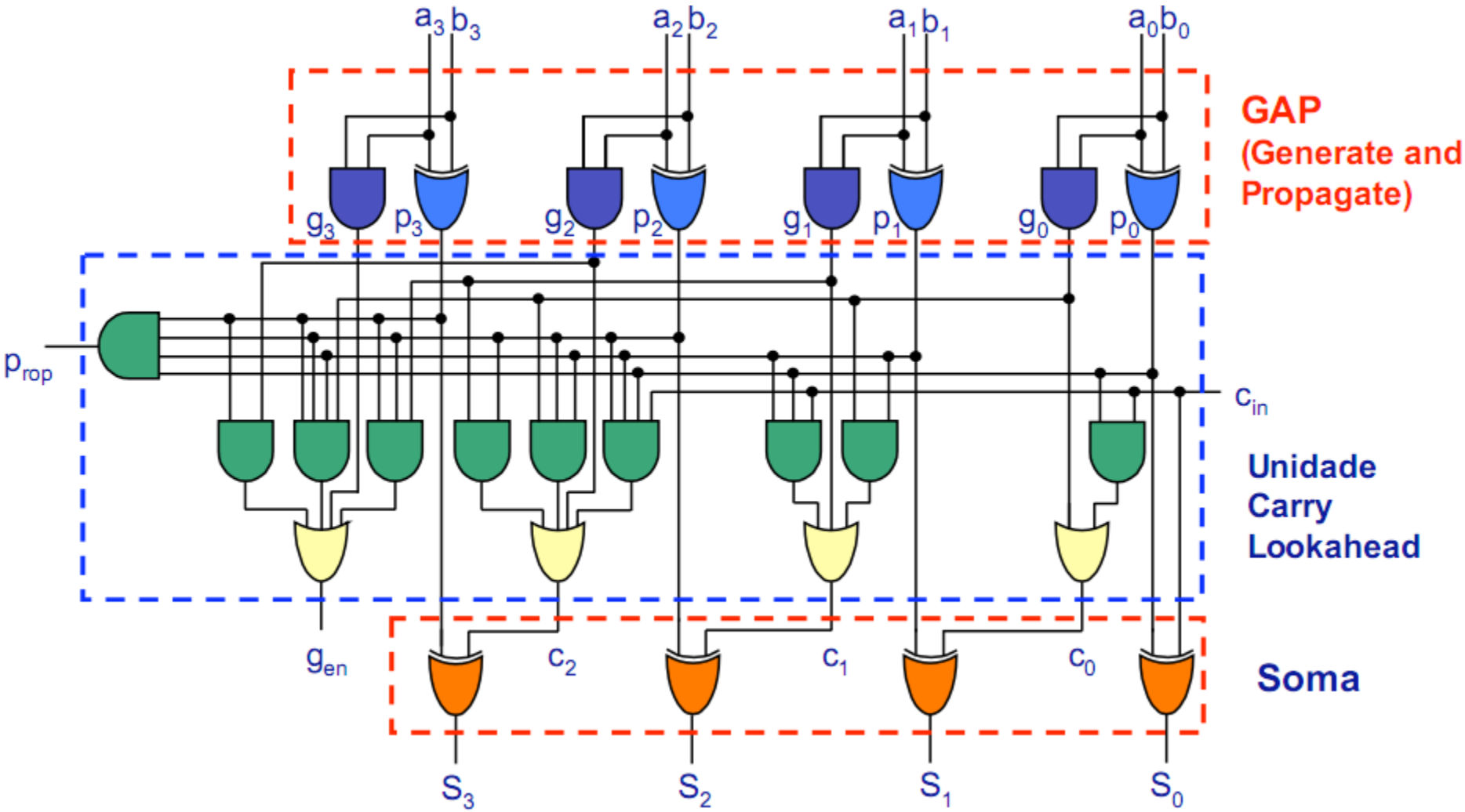
$$c_2 = g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0$$

$$c_3 = g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0$$

$$c_4 = g_3 + p_3 \cdot g_2 + p_3 \cdot p_2 \cdot g_1 + p_3 \cdot p_2 \cdot p_1 \cdot g_0 + p_3 \cdot p_2 \cdot p_1 \cdot p_0 \cdot c_0$$

Circuitos Aritméticos

Carry Lookahead



Circuitos Aritméticos

Carry Lookahead

- Problema com as unidades CLs: a complexidade da equação do *carry* cresce muito rapidamente!!!

- Para somadores com entradas usando muitos bits, a propagação do *carry* com cadeia *carry-lookahead* é mais lenta que um *ripple carry*.

- Tipicamente, o *carry-lookahead* não é usado para somadores com entradas maiores que 4 bits

Circuitos Aritméticos

Carry Lookahead

- Solução: aplicar o lookahead em grupos de somadores com, no máximo, 4 bits.

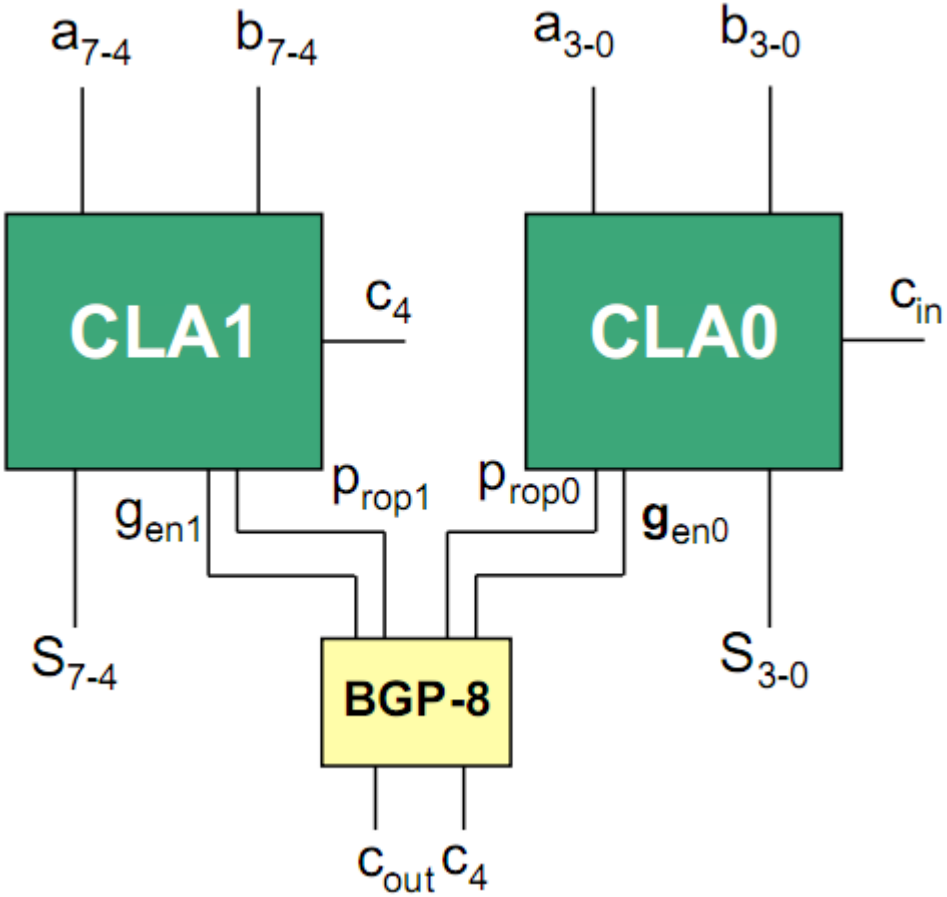
- Funções auxiliares P e G são necessárias e indicam, respectivamente:

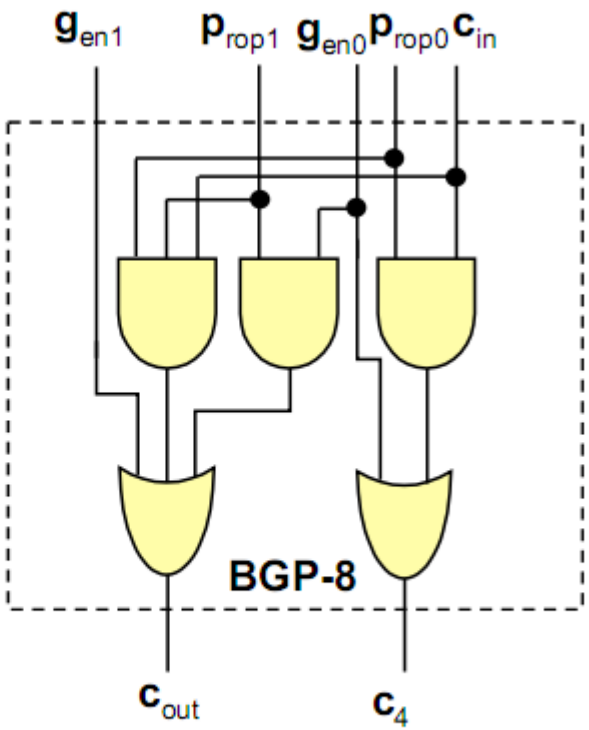
  Se P = 1, então o carry é propagado pelo grupo.

  Se G = 1, então o grupo gera carry out.

Circuitos Aritméticos

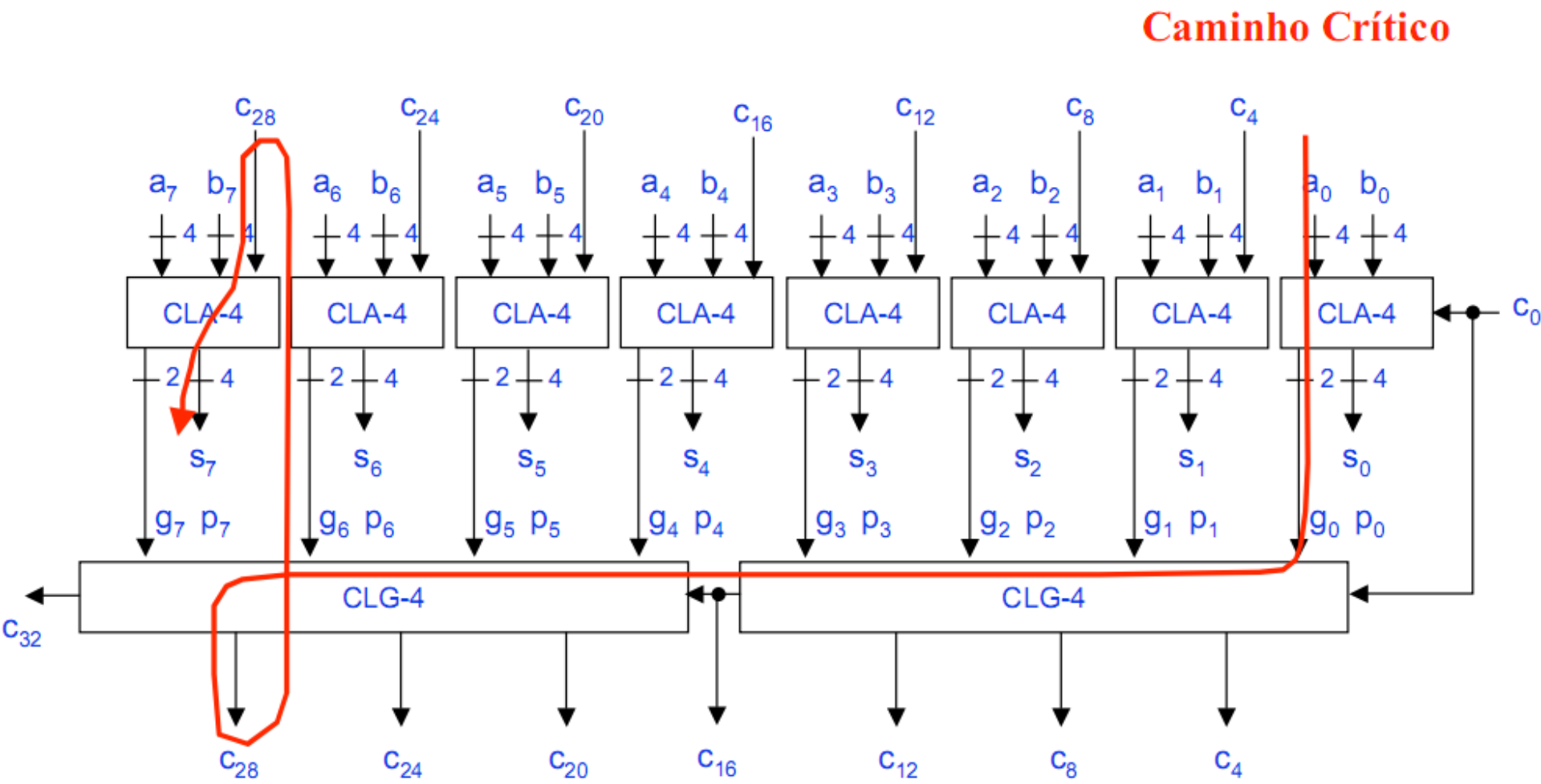
Carry Lookahead





Circuitos Aritméticos

Carry Lookahead



Carry Lookahead

Equações do CLG-4

Carry para o segundo bloco

$$C_4 = g_0 + p_0 C_0$$

Carry para o terceiro bloco

$$C_8 = g_1 + p_1 g_0 + p_1 p_0 C_0$$

Carry para o quarto bloco

$$C_{12} = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 C_0$$

Carry de saída do grupo

$$C_{16} = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 C_0$$

Circuitos Aritméticos

Carry Select

Princípios:

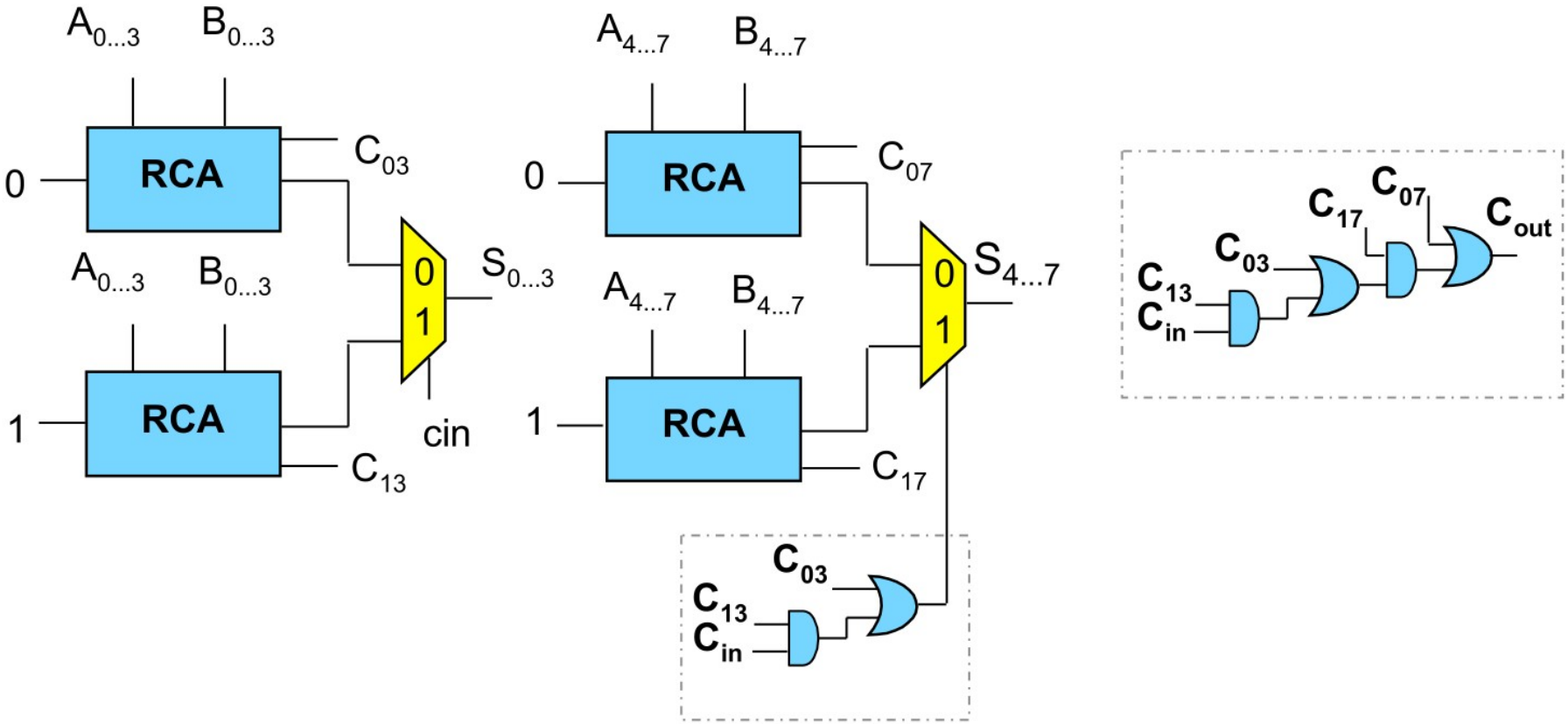
- **dividir a adição em seções de 4 ou 8 bits**
- **realizar a adição de cada seção simultaneamente para os dois casos possíveis (*carry in*=0 e *carry in*=1)**

- ✓ Em cada seção de adição são usados dois somadores (ripple carry ou carry lookahead) idênticos e um multiplexador
- ✓ O multiplexador seleciona um dos dois resultados, utilizando como controle o carry out da seção anterior

Circuitos Aritméticos

Carry Select

- Exemplo somador CSA de 8 bits



Carry Select

Equações dos MUX de cada seção da soma

B1 = Cin

B2 = C03 or (C13 and CIN)

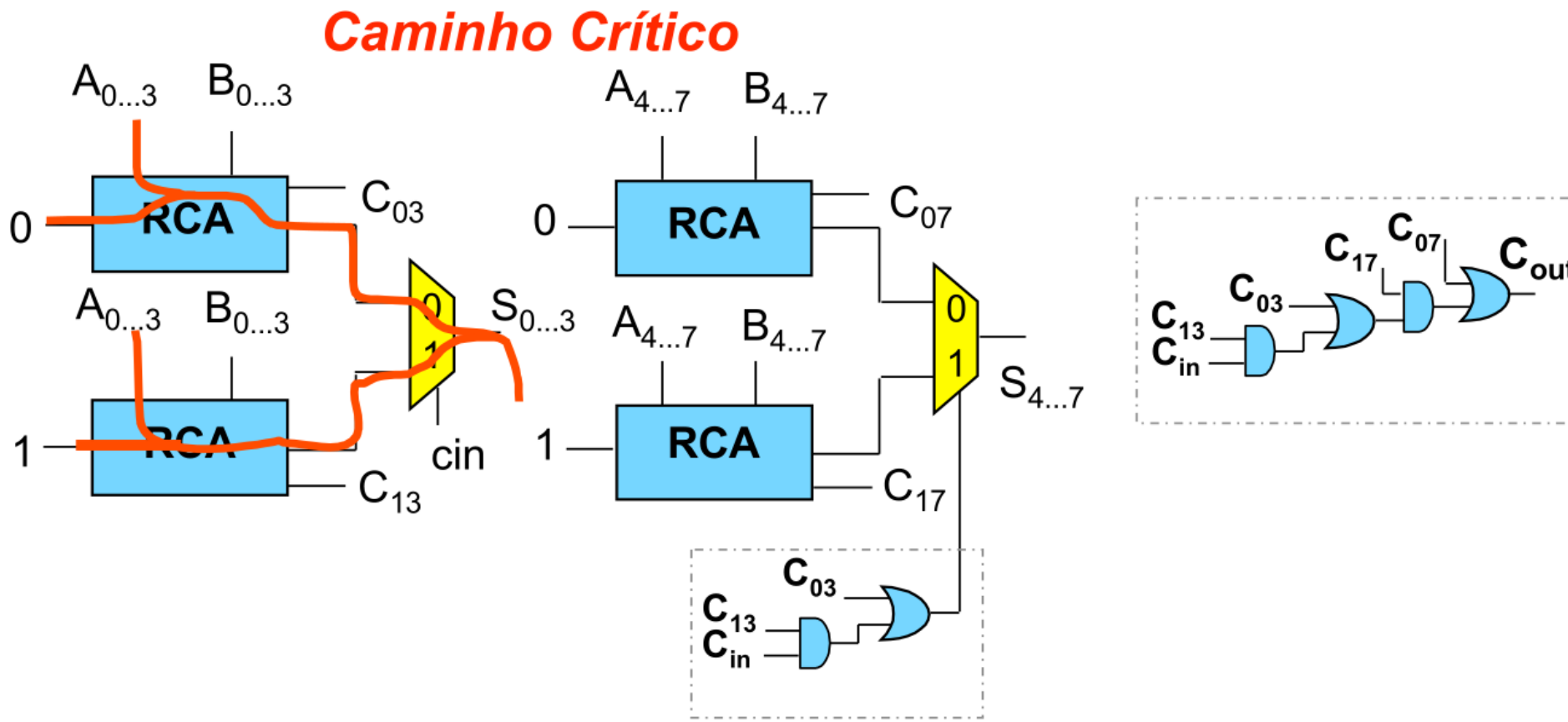
B3 = C07 or (C17 and (C03 or (C13 and CIN)))

B4 = C011 or (C111 and (C07 or (C17 and (C03 or (C13 and CIN)))))

B4 =C015 or (C115 and(C011 or (C111 and (C07 or (C17 and (C03 or (C13 and CIN))))))

Circuitos Aritméticos

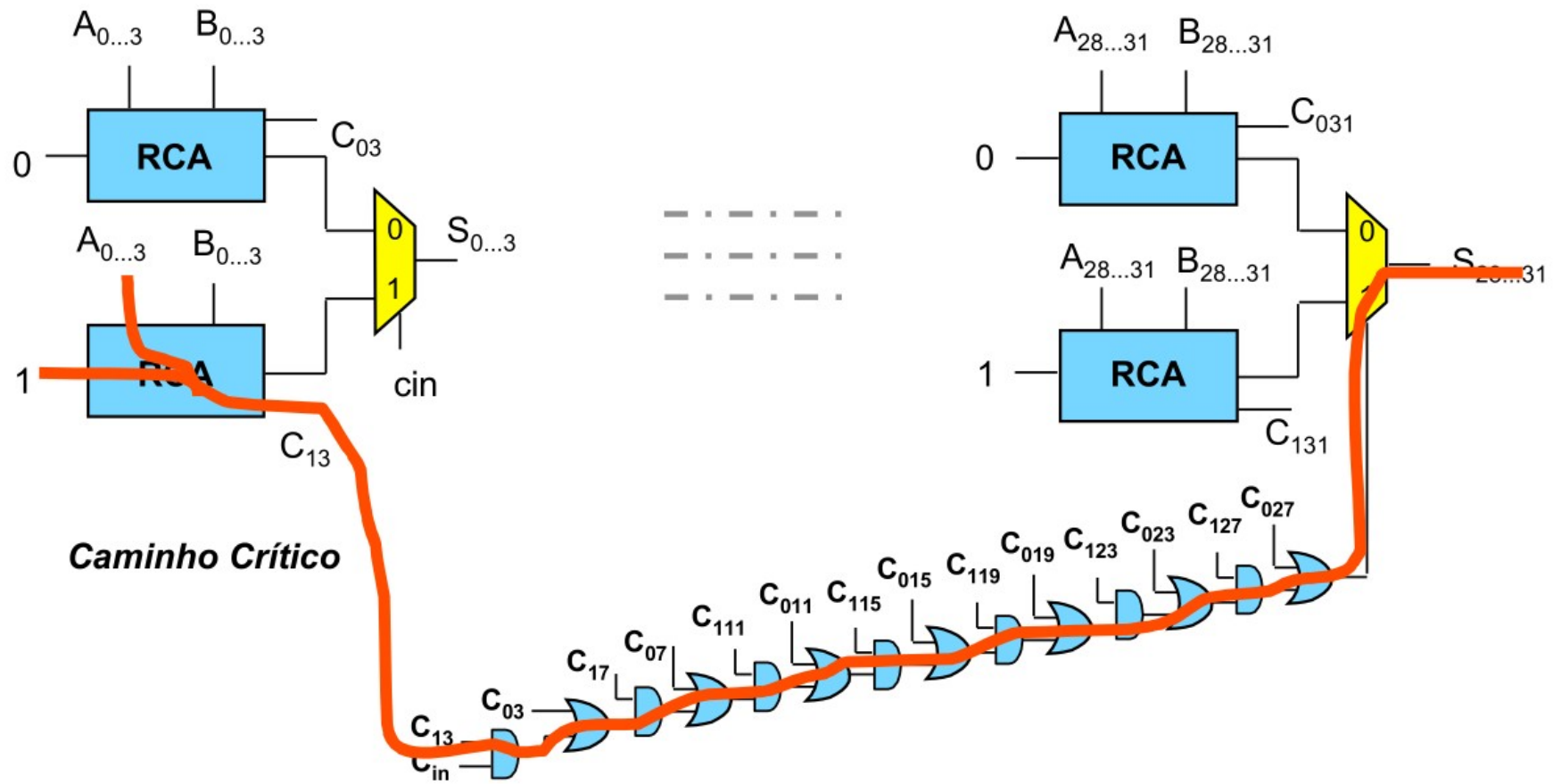
Carry Select 8 bits



Atraso crítico = Atraso RCA + Atraso MUX

Circuitos Aritméticos

▶ Carry Select Adder 32 bits



Atraso crítico = Atraso RCA + Atraso Equação + Atraso MUX